

進階監督式學習：集成、異常、關聯與時間序列

M4 — Ensembles, Anomalies, Association Rules & Time Series

M2 教你把單一模型做好。這一週處理四件單一模型做不到的事：

把模型組起來、找出沒有標籤的異常、挖出隱藏的關聯、處理有時間的資料。

本講義大綱

01 為什麼需要這一週 · 單一模型的四個天花板

02 偏誤—變異分解 · 為什麼集成有效

03 集成方法完整分類 · 平行·序列·異質

04 多樣性原理 · 成員必須「錯得不一樣」

05 Bagging 家族 · 隨機森林與 Extra Trees

06 Boosting 家族 · AdaBoost 手算三輪

07 GBDT 三大實作 · XGBoost · LightGBM · CatBoost

08 Stacking · 用模型組合模型

09 集成的代價 · 什麼時候不該用

10 異常有三種型態 · 點·情境·集體

11 異常偵測方法分類 · 統計·密度·模型·監督式

12 LOF 與 Isolation Forest · 兩個主力演算法

13 無標籤怎麼評估 · 異常偵測最大的難題

14 關聯規則三指標 · 以及 lift 陷阱

15 Apriori 與 FP-Growth · 兩種頻繁項目集演算法

16 醫療應用：共病與處方 · 購物籃分析的延伸

17 時間序列基礎 · 切分、滯後特徵、分解

18 機率校準 · 模型說 70% 真的是 70% 嗎

19 代價敏感學習 · 錯在哪一邊代價不同

20 AI Studio 實作 · 四個主題各一個範例

為什麼要有這一週：單一模型的四個天花板（上）

① 再怎麼調參，單一模型就是有極限

- ▶ 決策樹調到最好，仍然不穩定：換一批資料，樹就長得完全不同
- ▶ 邏輯斯迴歸再怎麼加特徵，也畫不出複雜的非線性邊界
- ▶ ★ 這時要換的不是「更強的模型」，而是「用一群模型」

② 有些問題根本沒有標籤

- ▶ 異常事件、設備故障、詐欺通常只有少量、延遲或不完整的異常標籤
- ▶ 標註夠多時，就改用監督式分類（M2-2）
- ▶ ★ 標註稀少時，M2 那套監督式方法很難直接套用

這兩點對應本模組的 PART 1-3：集成與異常偵測。

為什麼要有這一週：單一模型的四個天花板（下）

③ 有些問題不是在「預測」，而是在「發現」

- ▶ 「哪些藥常常一起開？」「哪些診斷常常同時出現？」
- ▶ 沒有 Y 可以預測，你要的是資料裡的結構
- ▶ 這類問題屬於關聯規則的範疇

④ 有些資料有時間順序，不能亂切

- ▶ ★ 用未來的資料預測過去，是資料科學最常見也最嚴重的錯誤之一
- ▶ M2 用的 Split Data 隨機（shuffled）切分，在這裡是災難
- ▶ 時間序列需要完全不同的切分與驗證方式

這兩點對應 PART 4-5：關聯規則與時間序列。四者的共同點是「換一種看資料的角度」。

四大主題的選用地圖

這張圖建議印出來貼著；本模組所有內容都掛在這四支上



★ 這四個主題的共同點：都不是「換一個更強的模型」，而是「換一種看資料的角度」。

先問「我手上的問題長什麼樣」，再決定往哪一支走——走錯支線，方法再強也沒有用。

有標註、想更準

集成方法。已經有模型了，想再榨出幾個百分點。

沒標註、想找怪的

異常偵測。設備監測、異常事件、品質稽核。

想找「常一起出現」

關聯規則。處方組合、共病分析、衛材耗用。

資料有時間順序

時間序列。★ 切分方式與前三者完全不同。

PART 01

集成為什麼有效

Why Ensembles Work

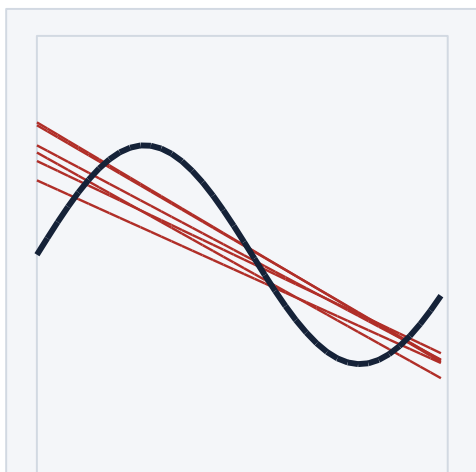
在學怎麼用之前，先弄懂它在解什麼問題。

這一節的觀念決定你之後會不會用錯。

偏誤—變異分解：集成到底降低了什麼

高偏誤・低變異

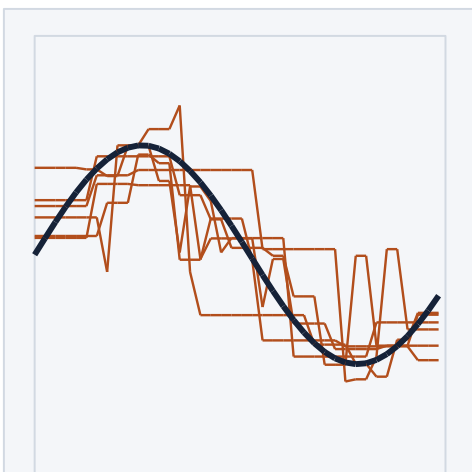
模型太簡單（單一淺樹）



欠擬合：怎麼抽樣都畫成同一條直線，離真相很遠

低偏誤・高變異

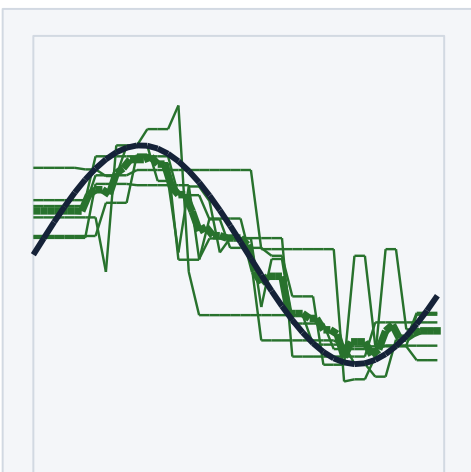
模型太複雜（單一深樹）



過擬合：每次抽樣結果差很多，抓到的是雜訊

低偏誤・低變異

集成（多棵樹平均）



★集成的目標：保留彈性，但把抖動平均掉

—— 真實關係

—— 六次不同抽樣訓練出的模型（細線）

—— 六個模型的平均（集成）

三張圖的真實關係（深色粗線）完全相同，差別只在模型的複雜度。左邊怎麼抽樣都畫成同一條直線＝偏誤大；中間每次抽樣差很多＝變異大；右邊把中間那六條平均起來，抖動互相抵消，就回到真實關係附近。

偏誤 Bias

模型太簡單，系統性地偏離真相。
表現：訓練與測試都不好（欠擬合）。

變異 Variance

模型太複雜，對訓練資料的抖動過度敏感。
表現：訓練很好、測試很差（過擬合）。

★ 一句話記住

Bagging 常藉平均降低變異；Boosting 常先改善偏誤，但正則化、樹深與資料雜訊也會影響兩者。

怎麼用這句話

模型不穩定（換資料結果差很多）→ 用 Bagging。
模型太弱（怎麼調都不準）→ 用 Boosting。

先診斷再開藥：三種症狀對應三種處方

症狀	訓練分數	測試分數	診斷	處方
兩邊都差	0.68	0.66	高偏誤（欠擬合）	★ 換更強的模型、加特徵、用 Boosting
訓練好測試差	0.99	0.71	高變異（過擬合）	★ 減少複雜度、加正則化、用 Bagging / 隨機森林
兩邊都好但不穩	0.91	0.88 （±0.09）	變異偏高	集成可降低波動；同時檢查資料量是否太少
兩邊都好且穩定	0.91	0.89 （±0.02）	健康	不需要集成。★ 別為了用而用

★ 「測試分數 ±」是交叉驗證各折的標準差。只看平均值看不出穩不穩定——這是最常被忽略的一欄。

集成的前提：成員必須「錯得不一樣」

三個模型都犯一樣的錯

第 1 - 10 題

模型 1	○	○	×	○	○	×	○	○	×	○	70%
模型 2	○	○	×	○	○	×	○	○	×	○	70%
模型 3	○	○	×	○	○	×	○	○	×	○	70%
多數決	○	○	×	○	○	×	○	○	×	○	70%

三個模型錯在同樣的第 3、6、9 題
多數決結果與單一模型完全相同 → 集成白做了

集成後仍是 70%

○ 答對

× 答錯

三個模型犯不同的錯

第 1 - 10 題

模型 1	○	○	×	○	○	×	○	○	×	○	70%
模型 2	○	×	○	○	○	○	×	○	○	○	80%
模型 3	×	○	○	○	○	○	○	×	○	○	80%
多數決	○	○	○	○	○	○	○	○	○	○	100%

每個模型錯的題目都不一樣
★ 多數決後 10 題全對：70% → 100%

★ 集成後 100%

成員都在 70-80%，差別在「錯在哪幾題」

★ 集成能不能變強，取決於成員「錯得不一樣」——這叫多樣性（Diversity）。

三種手段：抽不同的樣本（Bagging）、抽不同的特徵（Random Forest）、用不同演算法（Voting / Stacking）。

左邊：三個模型犯一樣的錯

多數決之後準確率完全沒變。集成白做了。

右邊：三個模型犯不同的錯

多數決之後 70% → 100%。

★ 這叫多樣性 Diversity

集成能不能變強，取決於成員錯得夠不夠不一樣。

製造多樣性的三種手段

抽不同樣本（Bagging）、抽不同特徵（Random Forest）、用不同演算法（Voting / Stacking）。

多樣性與準確度的取捨：集成的核心張力

成員要夠準，但也要夠不一樣

- ▶ 全部都很準但完全一樣 → 集成沒有增益（左圖的情況）
- ▶ 全部都很不一樣但都很爛 → 多數決也是爛的
- ▶ ★ 理想：每個成員都「比隨機猜好一點」，且彼此錯得不一樣

為什麼弱學習器（weak learner）反而好用

- ▶ Boosting 刻意用很淺的樹（甚至只有一層，叫決策樁 Decision Stump）
- ▶ 淺樹很弱，但每棵專注在不同的錯誤上 → 多樣性天然就高
- ▶ ★ 這解釋了一個反直覺的現象：多個較弱但互補的模型，可以組成表現很強的模型

怎麼檢查你的集成有沒有多樣性

- ▶ 看成員之間的預測相關性：相關係數接近 1 就表示白做工
- ▶ 看各成員的錯誤是不是集中在同一批樣本上
- ▶ 實務簡易法：拿掉任一成員，整體分數幾乎不變 → 那個成員是多餘的

PART 02

集成方法的完整分類

Taxonomy of Ensemble Methods

三大家族、十幾種方法，但分辨它們只需要問一個問題。

集成方法完整分類圖



平行 Parallel

各成員獨立訓練，互不知道彼此 → 降低變異。

Bagging、隨機森林、Extra Trees。

序列 Sequential

後者專門修正前者的錯 → 降低偏誤。

AdaBoost、Gradient Boosting、XGBoost 家族。

異質 Heterogeneous

組合不同型態的模型。

Voting、Stacking。

★ 一句話分辨

問「第 N 個模型訓練時，需不需要知道前 N-1 個的結果？」

不需要 = 平行；需要 = 序列；只用它們的預測值 = Stacking。

Bagging 家族：三種方法的差別

方法	抽樣本	抽特徵	分裂點選擇	特點
Bagging	★ 有放回抽樣 (Bootstrap)	全部使用	找最佳分裂點	最基本的形式；可搭配任何模型
隨機森林 Random Forest	★ 有放回抽樣	★ 每次分裂隨機抽 部分特徵 (sklearn $\sqrt{p}$ ； AI Studio 預設 $\log(p)+1$)	在抽到的特徵中找最佳	多樣性更高；分類問題的預設起點
Extra Trees	通常用全部資料	每次分裂隨機抽	★ 分裂點也隨機取， 不找最佳	更快、變異更低，但偏誤略高

★ 注意隨機森林的兩層隨機：抽樣本 + 抽特徵。第二層才是它比單純 Bagging 強的原因——它強迫每棵樹用不同的特徵，多樣性大增。

Bootstrap 抽樣：那 36.8% 是怎麼來的

有放回抽樣的數學

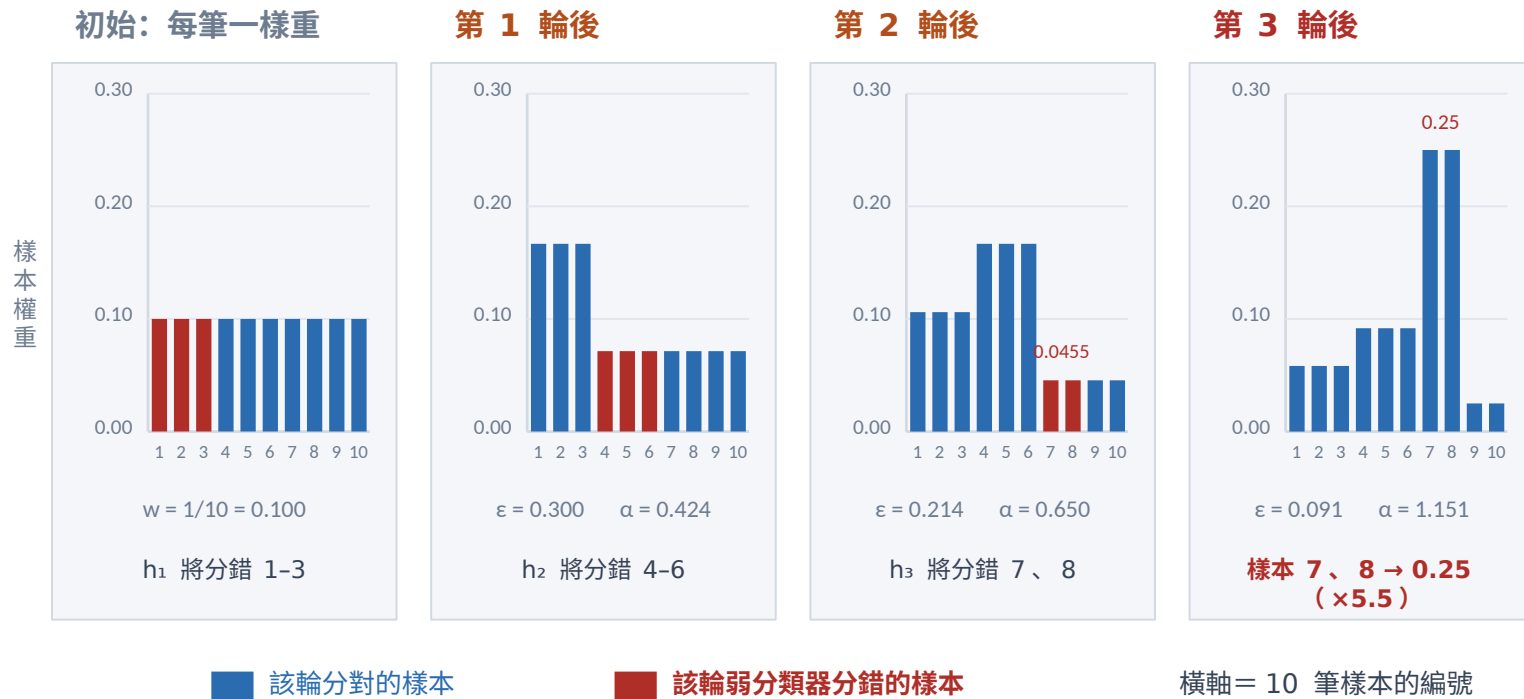
- ▶ n 筆資料中抽 n 次（有放回），某一筆從沒被抽到的機率：
- ▶ $(1 - 1/n)^n \rightarrow$ 當 n 夠大時趨近 $1/e \approx 0.368$
- ▶ ★ 所以平均約有 36.8% 的資料從未進入某棵樹的訓練集
- ▶ 這批資料就叫「袋外樣本」（Out-of-Bag, OOB）

OOB 的實用價值

- ▶ ★ 免費的驗證集：不用切出 validation set 就能評估
- ▶ 每棵樹用「它沒看過的那 36.8%」評估，再平均
- ▶ 資料量少的時候特別有用——不必犧牲資料去做驗證
- ▶ AI Studio 的 Random Forest 運算子可直接輸出 OOB 誤差

※ 驗算： $n=100$ 時 $(1-0.01)^{100} = 0.366$ ； $n=1000$ 時 $= 0.3677$ ； $n \rightarrow \infty$ 時 $= 1/e = 0.3679$ 。

AdaBoost 實際跑三輪：權重怎麼變



ϵ 逐輪下降 (0.300 $\rightarrow$ 0.214 $\rightarrow$ 0.091), α 逐輪上升 (0.424 $\rightarrow$ 0.650 $\rightarrow$ 1.151) —— 越準的分類器票數越重。但被反覆放大的那兩筆若其實是標錯的資料, AdaBoost 就會越陷越深。

每一輪做兩件事

- ① 訓練一個弱分類器
- ② 把它分錯的樣本權重調高

分類器權重 α

$\alpha = 0.5 \times \ln((1-\epsilon)/\epsilon)$ 。錯誤率越低, α 越大, 最終投票的話語權越重。

★ 看第 3 輪

樣本 7、8 的權重從 0.0455 放大到 0.25 (5.5 倍)。下一個分類器會被迫專心對付它們。

為什麼降低偏誤

每一輪都在補前面的不足, 整體逐步逼近真相。
代價是對雜訊與離群值敏感。

AdaBoost 手算：完整三輪（數值已驗算）

10 筆樣本，每輪一個弱分類器

手算（已用 Python 驗算）

【初始】每筆樣本權重 $w = 1/10 = 0.100$

【第 1 輪】弱分類器 h_1 分錯樣本 {1, 2, 3}

加權錯誤率 $\varepsilon_1 = 0.1 + 0.1 + 0.1 = 0.300$

分類器權重 $\alpha_1 = 0.5 \times \ln((1-0.300)/0.300) = 0.5 \times \ln(2.333) = 0.4236$

更新權重：分錯者 $\times e^{\alpha_1}$ 、分對者 $\times e^{-\alpha_1}$ ，再正規化

→ 分錯的三筆： $0.100 \rightarrow 0.1667$ （放大 1.67 倍）

→ 分對的七筆： $0.100 \rightarrow 0.0714$

【第 2 輪】 h_2 分錯樣本 {4, 5, 6}

$\varepsilon_2 = 0.0714 \times 3 = 0.2143$ （用「更新後」的權重算）

$\alpha_2 = 0.5 \times \ln((1-0.2143)/0.2143) = 0.6496$

→ 分錯的三筆： $0.0714 \rightarrow 0.1667$ （放大 2.33 倍）

【第 3 輪】 h_3 分錯樣本 {7, 8}

$\varepsilon_3 = 0.0455 \times 2 = 0.0909$

$\alpha_3 = 0.5 \times \ln((1-0.0909)/0.0909) = 1.1513$

→ 分錯的兩筆： $0.0455 \rightarrow 0.2500$ （放大 5.50 倍）

【最終預測】 $H(x) = \text{sign}(0.4236 \cdot h_1(x) + 0.6496 \cdot h_2(x) + 1.1513 \cdot h_3(x))$

★ 觀察三件事：

- ① ε 逐輪下降是本例刻意設計；實務上題目越來越難， ε 常回升——整體好壞要看 $H(x)$ 的誤差
- ② α 逐輪上升（ $0.424 \rightarrow 0.650 \rightarrow 1.151$ ）——越準的分類器票數越重
- ③ 但若資料中有標錯的樣本，它會被一路放大 → 這是 AdaBoost 的弱點

Boosting 家族的演進：每一代解決了什麼

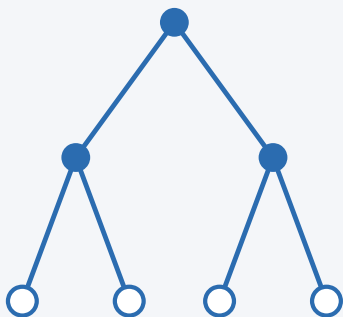
方法	核心作法	解決了什麼	留下什麼問題
AdaBoost 1997	調整「樣本權重」，讓後續分類器專注在錯誤上	讓「弱可組成強」（Schapire 1990 的理論）變得實用	★ 對雜訊與標錯的資料極敏感
Gradient Boosting 2001	改成擬合「殘差」（前面模型的誤差）	可用於任意可微損失函數；迴歸分類皆適用	訓練慢，且容易過擬合
XGBoost 2014	加入正則化項、二階導數、平行化	★ 速度與精度大幅提升，成為競賽主流	記憶體用量大
LightGBM 2017	逐葉生長、直方圖分箱、GOSS 抽樣	★ 大資料快很多倍	小資料容易過擬合
CatBoost 2018	Ordered Boosting、原生處理類別特徵	★ 類別型特徵不用自己做編碼，且避免目標洩漏	訓練較慢

★ 醫療資料常有大量類別型特徵（科別、診斷碼、藥品碼），CatBoost 往往是很好的預設選擇。AI Studio 內建 Gradient Boosted Trees；LightGBM、CatBoost 需透過 Python。

GBDT 三大實作的分裂策略

XGBoost

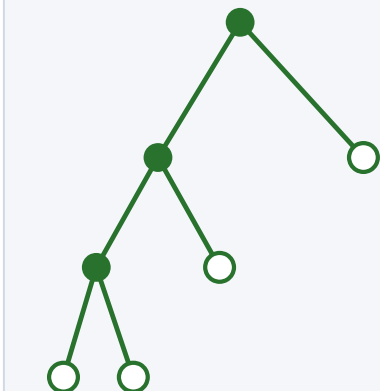
逐層生長 Level-wise



整層一起長，樹形對稱
穩定、不易過擬合，但會長出沒用的分支

LightGBM

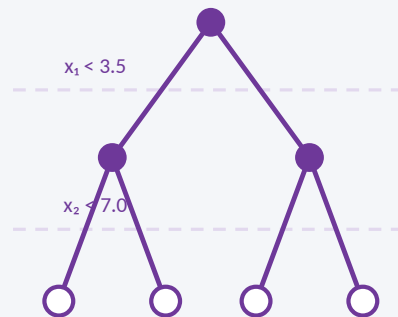
逐葉生長 Leaf-wise



一路往這邊深長
★ 每次挑「增益最大」的葉子往下長
同樣葉數下更準更快，小資料易過擬合

CatBoost

對稱樹 Symmetric



同一層用同一個切分條件
預測極快，對類別型特徵處理最好

● 實心＝分裂節點

● 空心圈＝葉節點

三棵樹的葉數相同，長法不同

同樣是梯度提升，差別只在「下一刀切哪裡」。逐層生長把整層補滿（對稱但浪費）；逐葉生長只挑增益最大的那片葉子（不對稱但有效率）；對稱樹整層共用一個條件（預測最快）。

XGBoost 逐層生長

整層一起長，樹形對稱。穩定、不易過擬合，但會長出沒用的分支。

★ LightGBM 逐葉生長

每次挑「增益最大」的葉子長。同樣葉數下更準更快，但小資料容易過擬合（要限制 max_depth）。

CatBoost 對稱樹

同一層用同一個切分條件。預測極快，且對類別型特徵處理最好。

選用建議

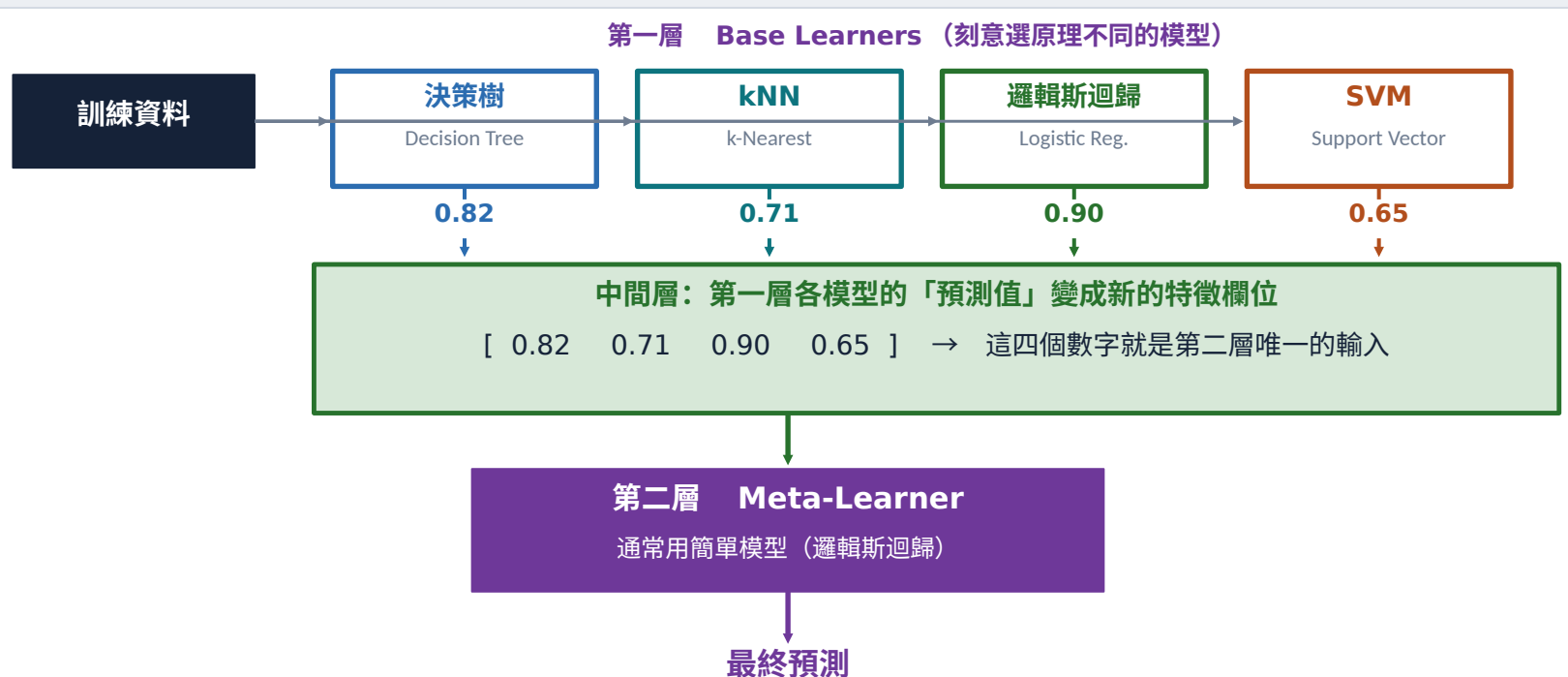
資料大又趕時間 → LightGBM；類別特徵多 → CatBoost；要穩定、文件齊全 → XGBoost。

Boosting 的關鍵參數：學習率與棵數的交易

參數	作用	調小會怎樣	調大會怎樣	建議起點
learning_rate 學習率	每棵樹的貢獻比例	更穩、更準，但需要更多棵樹	訓練快，但容易錯過最佳解	0.05 - 0.1
n_estimators 樹的數量	總共長幾棵	欠擬合	過擬合（但有早停就還好）	★ 設大一點 + 早停
max_depth 樹深	單棵樹的複雜度	欠擬合	過擬合、訓練慢	3 - 6（比隨機森林淺）
subsample 樣本抽樣比例	每棵樹用多少資料	多樣性高但可能欠擬合	接近 1 就失去隨機性	0.8
early_stopping 早停	驗證分數不再進步就停	—	—	★ 一定要開，這是最有效的防過擬合手段

★ 學習率與棵數是一組交易：學習率減半，棵數大致要加倍。實務做法是固定一個小學習率（0.05），然後用早停決定棵數。

Stacking：用一個模型來組合其他模型



★ 最關鍵的一件事

第一層的預測值必須用「交叉驗證的 **out-of-fold 預測**」產生，不能用它訓練過的資料。
否則第二層學到的是「第一層記住答案」這件事 → 嚴重資料洩漏，驗證分數虛高、上線就崩。
這也是 Stacking 最容易做錯的一步：流程對了才有價值，做錯比不用還糟。

第一層

數個不同型態的模型各自預測。
刻意選「原理不同」的模型以製造多樣性。

中間層

把第一層的預測值當成新的特徵欄位。

第二層 Meta-Learner

通常用簡單模型（邏輯斯迴歸）學「怎麼組合」。

★ 最關鍵的一件事

第一層的預測值必須用交叉驗證的 **out-of-fold 預測** 產生。
用它訓練過的資料 → 嚴重資料洩漏，上線就崩。

三種異質集成的比較

方法	怎麼組合	需要調什麼	什麼時候用
硬投票 Hard Voting	看誰的票多（多數決）	成員的選擇	成員表現接近、都輸出類別時
軟投票 Soft Voting	把各成員的機率平均	成員的選擇 + 權重	若機率可比較，常比硬投票更有效，因為保留信心程度
Stacking	訓練一個模型來學「怎麼組合」	成員 + meta 模型 + CV 折數	組合潛力高，但不保證最佳；複雜度與洩漏風險最高

★ 實務順序：先試軟投票（簡單、不易出錯）。只有在軟投票明顯不夠、且你有時間仔細處理 out-of-fold 時，才上 Stacking。

軟投票 vs 硬投票：一個會改變結論的差別

三個模型對同一位病人「30 天內再入院」的預測

模型	預測機率 P(再入院) 0 — 1	硬投票的票	信心程度
決策樹	 0.55	是	勉強過半
kNN	 0.52	是	勉強過半
邏輯斯迴歸	 0.10	否	非常確定

判定門檻 0.5

硬投票 Hard Voting

票數：是 2 票、否 1 票

→ 判定：是（會再入院）

★ 兩張「勉強過半」的票，蓋過了一張「非常確定」的票
支持比例 2/3

軟投票 Soft Voting

平均機率 = $(0.55 + 0.52 + 0.10) / 3 = 0.39$

→ 判定：否（不會再入院）

★ 保留了「有多確定」的資訊，結論完全相反
平均機率 0.39

★ 同一組模型、同一位病人，兩種投票法給出相反的答案。

實務建議：只要成員模型能輸出機率，優先用軟投票（前提是機率可比較）。硬投票丟掉的「信心程度」往往正是關鍵資訊。

硬投票

票數 2:1 → 判定「會再入院」。
★ 兩張「勉強過半」的票蓋過一張「非常確定」的票。

軟投票

平均機率 0.39 → 判定「不會」。結論完全相反。

★ 實務建議

只要成員能輸出機率，優先用軟投票。
硬投票丟掉的「信心程度」往往正是關鍵資訊。

但有前提

各成員的機率必須是「可比較的」——
這就帶到下一個主題：機率校準。

集成的代價：什麼時候不該用（上）

① 可解釋性大幅下降

- ▶ 單棵決策樹可以印出來給臨床看；100 棵樹不行
- ▶ ★ 醫療場域若需要逐案解釋判斷依據，集成會是障礙
- ▶ 折衷：用 SHAP 等事後解釋工具（M9 會談），但那是另一層近似

② 訓練與預測成本上升

- ▶ 訓練時間乘上成員數；預測時每一筆都要跑過所有成員
- ▶ 即時性要求高的場景（如床邊監測）要先確認延遲可接受

這兩點是「用了會付出什麼」。

集成的代價：什麼時候不該用（下）

③ 單一模型已經夠好的時候

- ▶ ★ 如果邏輯斯迴歸 AUC 0.88、集成 0.89，那 0.01 值得嗎？
- ▶ 多花的維運成本、失去的可解釋性，通常不值得
- ▶ 先問「這 0.01 會改變任何臨床決策嗎」

④ 資料本身有問題的時候

- ▶ 集成放大不了不存在的訊號。資料髒、標註爛、特徵沒訊息量——
- ▶ ★ 換再厲害的集成也救不回來。回去修資料，那才是投報率最高的事

★ 這兩點是「根本不該用」的情況。集成是工具不是目標，會用也要會判斷什麼時候不用。

實作 1 | Titanic 集成模型比較

【AI Studio 範例】 Ensemble - bagging vote stacking Adaboost.rmp

開啟檔案

Repository ▶ 3. Outlier and Ensemble/
Ensemble - bagging vote
stacking Adaboost.rmp
Titanic.rmhdf5table
WineData.rmhdf5table

★ 此範例一次示範四種集成，
是本模組最重要的檔案

預期結果

- ▶ Bagging 的標準差明顯小於單一決策樹
- ▶ AdaBoost 常能提升平均，但對雜訊敏感
- ▶ 軟投票通常略優於硬投票
- ▶ 哪種最好因資料而異；Stacking 訓練時間最長

操作步驟

- 1 先跑單一決策樹，記錄準確率與各折標準差（基準線）
- 2 跑 Bagging（10 個成員），比較準確率「與標準差」的變化
- 3 ★ 重點看標準差：Bagging 主要降的是變異，不一定拉高平均
- 4 跑 AdaBoost，觀察它在哪些樣本上反覆調權重
- 5 跑 Vote（決策樹 + kNN + 邏輯斯迴歸），比較硬投票與軟投票
- 6 跑 Stacking，注意 meta-learner 的設定
- 7 把五種結果做成一張表：準確率、標準差、訓練時間

PART 03

異常偵測

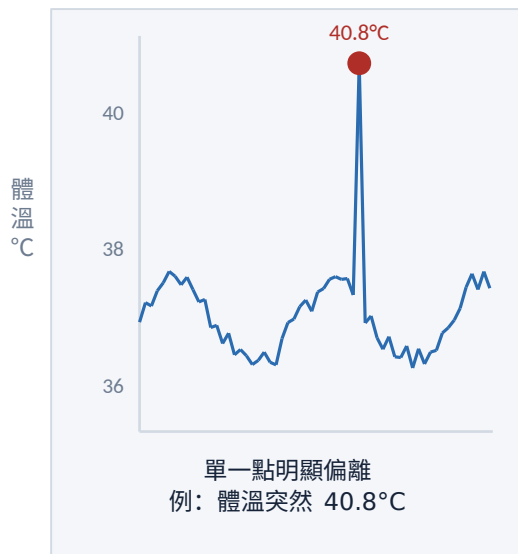
Anomaly Detection

當你沒有標籤，卻要找出「不對勁」的東西。
這是醫療與設備監測領域最實用的一類方法。

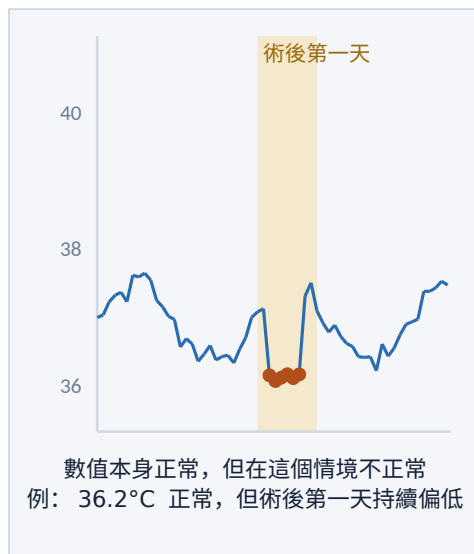
異常有三種型態，偵測方法完全不同

以體溫監測為例

① 點異常 Point



② 情境異常 Contextual



③ 集體異常 Collective



三張圖的橫軸都是「連續 120 小時」，縱軸都是體溫（°C）

★ 先分辨你要抓的是哪一種。多數人只想到①，但臨床上②③往往更重要，也更難抓。

①只要看單點數值；②必須先建立「情境」（術後第幾天、哪個班別）；③必須看一整段區間的統計特性。

① 點異常

單一點明顯偏離。例：體溫突然 40.8°C。最容易抓，統計法就夠。

② 情境異常

數值本身正常，但在這個情境不正常。例：36.2°C 正常，但術後第一天持續偏低。

③ 集體異常

每一點都正常，但這一段整體不正常。例：連續 25 小時波動消失（訊號僵住）。

★ 先分辨你要抓的是哪一種

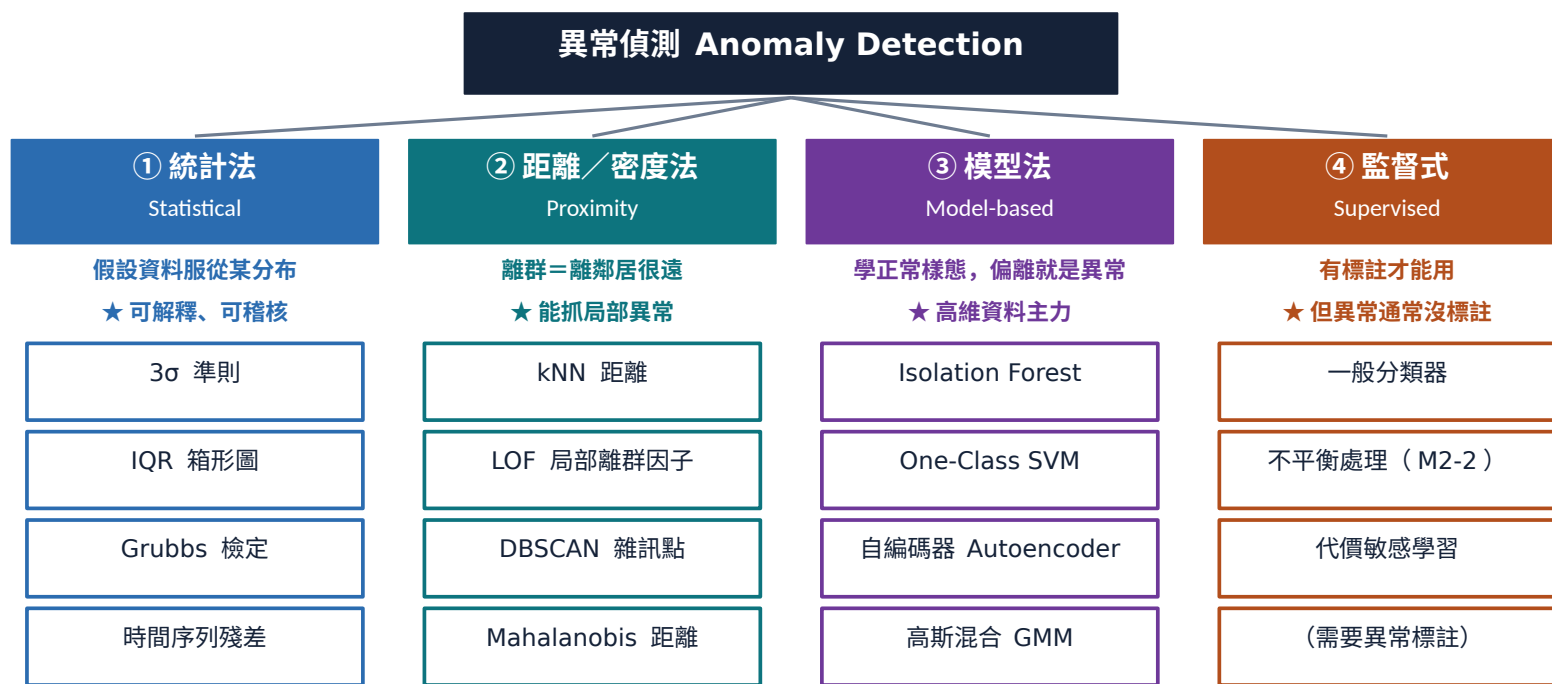
多數人只想到①，但臨床上②③往往更重要，也更難抓。

三種異常型態的偵測策略

型態	怎麼抓	需要什麼	醫護場域的例子
點異常	統計門檻（ 3σ 、IQR）、距離法	單一時間點的數值	檢驗值超出參考範圍、給藥劑量異常
情境異常	★ 先分群或建立情境模型，再在情境內比較	情境變數（時段、術後天數、年齡層）	夜班的活動量、術後第 N 天的引流量
集體異常	看一段區間的統計特性（變異數、自相關）	★ 序列資料，不是單點	監測訊號僵住、連續數小時無紀錄

★ 第二列是實務上最有價值也最常被做錯的：把所有病人的數值混在一起算門檻，情境異常就完全抓不到。

異常偵測方法的完整分類



★ 選用順序：先試統計法（最好解釋）→ 維度高或有局部結構就換密度法 → 都不行才用模型法

最右邊的監督式只有在「你已經有一批標好的異常案例」時才能用

—— 而那通常正是你沒有的東西。標註夠多時，就能改走監督式分類。

統計法

3 σ 、IQR、Grubbs 檢定。假設資料服從某分布。

★ 可解釋、可稽核——醫療場域的首選起點。

距離／密度法

kNN 距離、LOF、DBSCAN 雜訊點。

★ 能抓局部異常。

模型法

Isolation Forest、One-Class SVM、自編碼器。

★ 高維資料的主力。

★ 選用順序

先試統計法 → 維度高或有局部結構就換密度法 → 都不行才用模型法。

統計法：三種門檻的實際計算與適用時機

看起來簡單，但選錯方法會漏掉一大半

統計法（數值為示範用）

【資料】某病房 30 天的日均給藥次數

平均 $\mu = 48.2$ 標準差 $\sigma = 6.4$ 中位數 = 47.0 $Q1 = 43.5$ $Q3 = 52.0$

【方法一】 3σ 準則

上界 = $\mu + 3\sigma = 48.2 + 19.2 = 67.4$

下界 = $\mu - 3\sigma = 48.2 - 19.2 = 29.0$

✓ 適用：資料接近常態分布

✗ 陷阱：★ μ 與 σ 本身會被離群值拉走 —— 一筆極端值就能讓門檻失效

【方法二】IQR 箱形圖法（穩健，推薦預設）

$IQR = Q3 - Q1 = 52.0 - 43.5 = 8.5$

上界 = $Q3 + 1.5 \times IQR = 52.0 + 12.75 = 64.75$

下界 = $Q1 - 1.5 \times IQR = 43.5 - 12.75 = 30.75$

✓ 適用：★ 不假設特定分布，且較不受極端值影響（中位數與四分位數都穩健，但不是完全免疫）

✓ 這就是 M1 箱形圖上那兩條鬚

【方法三】穩健 Z 分數（MAD）

$MAD = \text{median}(|x_i - \text{median}|) = 4.5$

修正 $Z = 0.6745 \times (x - \text{median}) / MAD$ （0.6745 為常態一致化常數）

門檻通常取 $| \text{修正 } Z | > 3.5$

✓ 適用：★ 樣本數少、或分布明顯偏斜時最可靠

【怎麼選】

資料近常態且乾淨 → 3σ

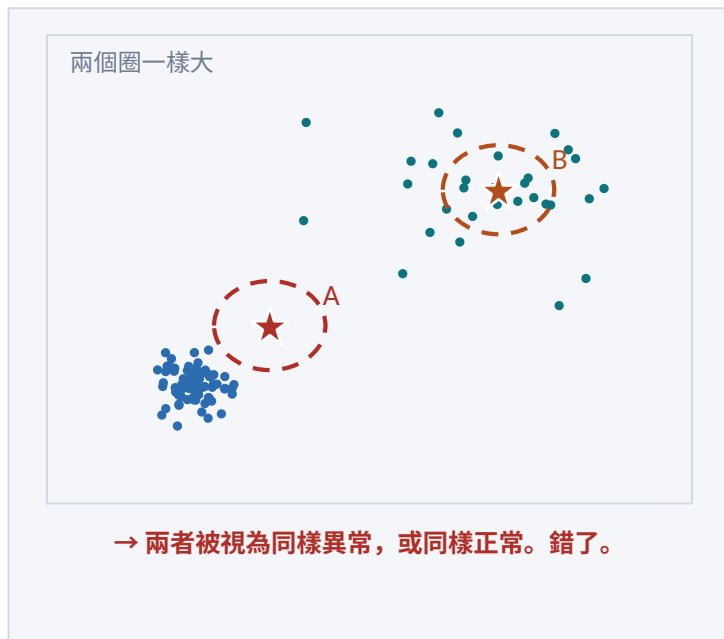
不確定分布 → ★ IQR（最安全的預設）

樣本少或嚴重偏斜 → MAD

LOF：為什麼「局部」兩個字這麼重要

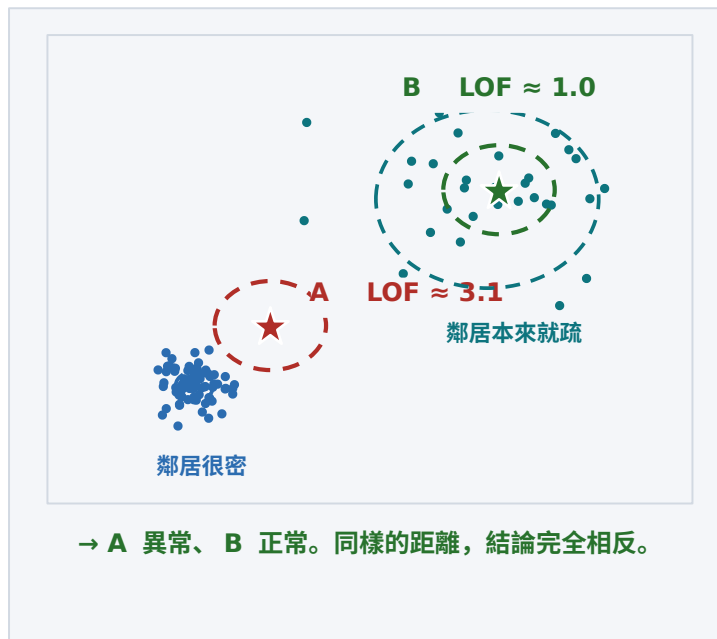
左・用「全域距離」判斷

A 與 B 到最近鄰的距離差不多



右・LOF：跟「鄰居的密度」比

A 週遭很密它卻疏 → 異常



● 密集群（80 點）

● 稀疏群（30 點）

★ 待判定的點 A、B

虛線圈＝鄰域半徑

LOF ≈ 1 表示與鄰居密度相當（正常）；明顯 > 1 表示比鄰居稀疏（異常）。注意：LOF 的「局部」是跟特徵空間裡的鄰居比；臨床上「跟病人自己平常比」是個人基線，概念相近但不相同。

全域距離的問題

A 與 B 到最近鄰的距離差不多 → 被視為同樣異常。錯了。

LOF 的作法

拿自己的密度和「鄰居的密度」比。
A 週遭很密它卻疏 → 異常；B 週遭本來就疏 → 正常。

怎麼讀數值

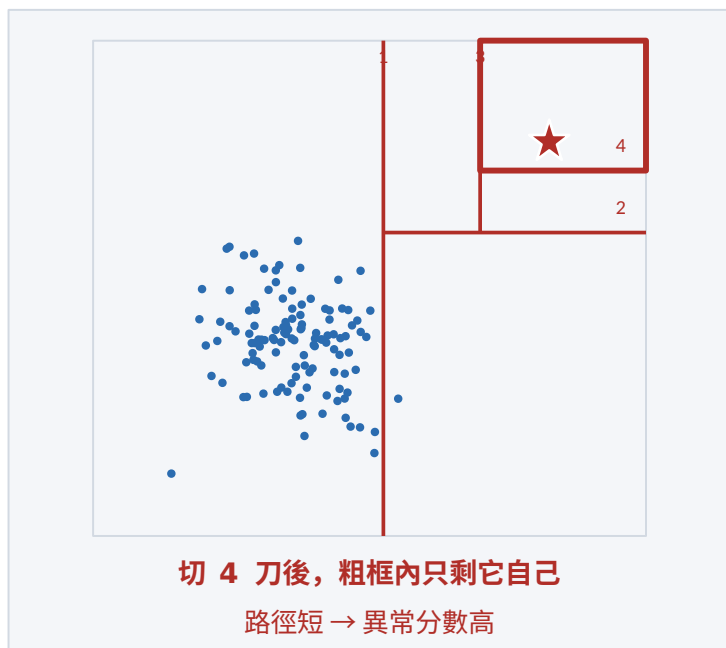
LOF ≈ 1 表示與鄰居密度相當（正常）；
明顯 > 1 表示比鄰居稀疏（異常）。

★ 醫療上的意義

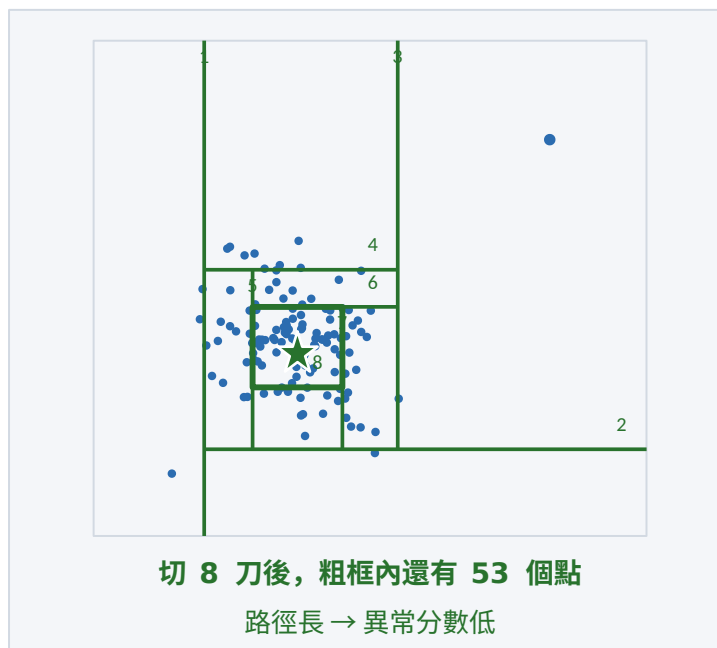
同一位病人的生理數值要跟「他自己平常」比，
不是跟全院平均比——這就是局部的概念。

Isolation Forest：用「切幾刀能孤立它」衡量異常

左・異常點：切 4 刀就被孤立



右・正常點：切 8 刀還在人群裡



● 其他資料點

★ 被追蹤的那個點

細線＝一刀（依序編號），粗框＝未切開區

建很多棵這樣的隨機切分樹，記錄每個點「平均要切幾刀才孤立得了」——這個刀數就是異常分數。不需要算距離、不需要假設分布，高維時仍然有效。

原理

隨機挑特徵、隨機挑切點，反覆切。
異常點很快就被單獨切出來，正常點要切很多刀。

★ 反直覺但高明的地方

它不去定義「什麼是正常」，
而是直接找「容易被孤立的點」。

為什麼適合高維

不需要算距離 → 較不受維度詛咒影響，速度快；無關特徵多時仍會變差。

關鍵參數

contamination（預期異常比例）——
★ 這個值你其實不知道，設定時要保守並搭配人工覆核（AI Studio 需 Anomaly Detection 擴充或 Python / PyOD）。

四種主流異常偵測演算法的對照

演算法	原理	維度	可解釋性	什麼時候用
IQR / 3σ	統計門檻	單變量	★ 極高	單一指標監測；一定要先試這個
LOF	局部密度比較	中低維	中	★ 資料有明顯的密度差異（不同群體混在一起）
Isolation Forest	隨機切分路徑長度	★ 高維	低	特徵較多、資料量大時的候選起點
One-Class SVM	學一個包住正常資料的邊界	中維	低	正常樣態很集中、且資料量不大時

※ 自編碼器（Autoencoder）屬深度學習取徑，M5 會談：訓練它重建正常資料，重建誤差大的就是異常。

異常偵測怎麼評估：沒有標準答案的時候

異常偵測最大的困難不是演算法，是「你怎麼知道抓對了」

- 沒有標準答案：如果你已經知道哪些是異常，就不需要異常偵測了
- 準確率算不出來：分母（真正的異常總數）你不知道

★ 這是它與分類問題最根本的差別 —— M2-2 那套評估指標在這裡多半用不上

① 人工覆核前 N 名

把模型分數最高的 50 筆交給專家看

算出「精確率 @50」
這是最務實的做法

② 注入已知異常

把已知的異常案例混進資料，看抓不抓得到

算出「召回率」
但只對這類異常有效

★ ③ 穩定性檢查

換參數、換時段跑，看抓到的是不是同一批

每次結果都不同
→ 代表模型不可信

★ 實務建議：把異常偵測定位成「幫人縮小檢視範圍的工具」，而不是「自動判定異常的系統」。

從 10 萬筆縮到 200 筆讓專家看，價值就已經很大了 —— 不必追求它能取代人。

★ 最大的困難不是演算法

是「你怎麼知道抓對了」。如果你已經知道哪些是異常，就不需要異常偵測了。

① 人工覆核前 N 名

把分數最高的 50 筆交給專家看，算「精確率 @50」。這是最務實的做法。

② 注入已知異常

把已知案例混進資料看抓不抓得到。可算召回率，但只對這類異常有效。

★ ③ 穩定性檢查

換參數、換時段跑，看抓到的是不是同一批。
每次結果都不同 → 模型不可信。

異常偵測的三個實務原則

① 定位成「縮小範圍的工具」，不是「自動判定系統」

- ▶ 從 10 萬筆縮到 200 筆讓專家看，價值就已經很大了
- ▶ ★ 不必追求它能取代人——那既做不到，責任上也不該這樣設計
- ▶ 在醫療場域這個定位還能大幅降低法規與責任風險

② 門檻由「人力可負荷量」決定，不是由統計決定

- ▶ ★ 專家一天能看 50 筆，那門檻就設在每天產出約 50 筆的位置
- ▶ 設得太嚴 → 漏掉真正的問題；設得太鬆 → 沒人看，系統形同虛設
- ▶ 這是統計課本不會教、但實務上最重要的一件事

③ 一定要留回饋迴路

- ▶ 專家覆核的結果要能回寫成標註
- ▶ 累積一段時間後，你就有了監督式學習的訓練資料
- ▶ ★ 這是從「異常偵測」演進到「分類模型」的正確路徑

實作 2 | 鍋爐感測器異常偵測

【AI Studio 範例】 Anomaly Detection .rmp、Boiler Sensor 資料集

開啟檔案

Repository ▶ 3. Outlier and Ensemble/
Anomaly Detection .rmp
Anomaly - outlier detection.rmp
Boiler Sensor.rmhdf5table

★ 感測器資料的結構與
醫療監測訊號高度類似，
方法可直接移轉

預期結果

- ▶ IQR 只抓得到單點極端值
- ▶ LOF 能抓到「數值不極端但脈絡奇怪」的點
- ▶ contamination 設定直接決定抓幾筆——它不是自動的
- ▶ 三種方法的交集通常才是最可信的異常

操作步驟

- 1 先畫時序圖，用肉眼找出你認為的異常段落（這只是初步候選，不等於客觀真值）
- 2 跑 IQR 門檻，看抓到哪些——通常只抓到「點異常」
- 3 跑 LOF，比較它抓到的與 IQR 有什麼不同
- 4 跑 Isolation Forest，把 contamination 分別設 0.01 / 0.05 / 0.10
- 5 ★ 比較三個設定抓到的是不是同一批（穩定性檢查）
- 6 把三種方法的結果做成一張表：各抓幾筆、重疊多少
- 7 回頭對照你第一步的標註，算出「相對於你標註」的精確率

PART 04

關聯規則

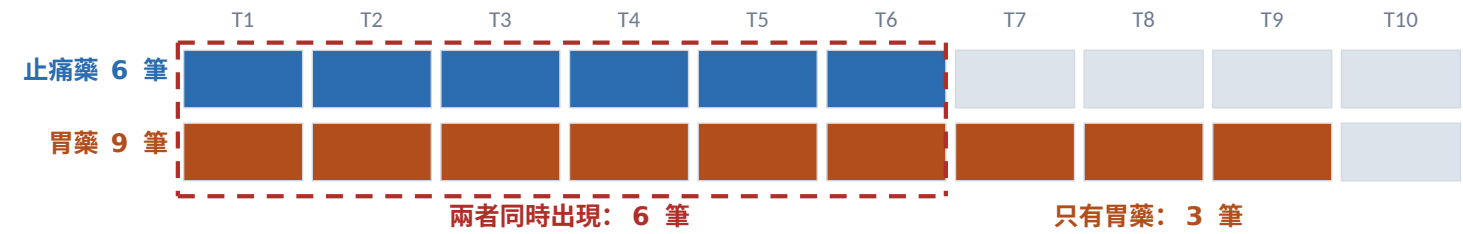
Association Rule Mining

沒有 Y 可以預測，你想知道的是「什麼跟什麼常常一起出現」。
購物籃分析出身，但在醫療領域用途更廣。

三個指標：為什麼信賴度會騙人

規則： { 止痛藥 } → { 胃藥 }， 10 筆處方交易

10 筆處方交易的實際內容 (■ 表示這筆交易裡有這個藥)



支持度 Support 這條規則涵蓋多少比例的交易 $6 / 10 = 0.60$ 太低表示組合太罕見，不值得管

信賴度 Confidence 買了止痛藥的人，有多少也買胃藥 $6 / 6 = 1.00$ 看起來完美，但先別下結論

提升度 Lift 比「隨機碰巧一起出現」強多少倍 $1.00 / 0.90 = 1.11$ ★ 只比隨機好 11%，沒什麼特別

為什麼 lift 才是關鍵

胃藥本來就出現在 9/10 的交易裡。
隨便挑一個藥，它的「信賴度」都會很高，
因為胃藥幾乎人人都開。

lift = 1 → 兩者獨立，沒有關聯
lift > 1 → 正相關 (越大越值得注意)
lift < 1 → 負相關 (互斥)

支持度 Support

這條規則涵蓋多少比例的交易。
太低表示組合太罕見，不值得管。

信賴度 Confidence

買了 A 的人有多少也買 B。
本例 $6/6 = 1.00$ ，看起來完美。

★ 提升度 Lift

比「隨機碰巧一起出現」強多少倍。
本例僅 1.11—— 只比隨機好 11%。

為什麼

胃藥本來就出現在 9/10 的交易裡。
隨便挑一個藥，信賴度都會很高。

lift 陷阱：信賴度排行榜為什麼會誤導你

同一份資料跑出來的六條規則，依信賴度由高到低排序

規則 (依 confidence 排序)	support	confidence	lift (虛線 = 1.0)	判讀
止痛藥 → 胃藥	0.60	1.000	1.111	信賴度 100%，但 lift 僅 1.11
降血壓藥 → 胃藥	0.30	0.750	0.833	★ 陷阱：信賴度高，lift 卻 < 1
維他命 → 胃藥	0.20	0.667	0.741	★ 同上，實際是負相關
胃藥 → 止痛藥	0.60	0.667	1.111	與第一條同一組關聯，方向不同
胃藥 → 降血壓藥	0.30	0.333	0.833	低信賴度也低 lift
胃藥 → 維他命	0.20	0.222	0.741	同上

★ 只用信賴度排序，你會把虛線框裡那兩條當成重要發現 —— 但它們的 lift 都在 1.0 的左邊。

★ 這兩條規則實際上是「負相關」：開了降血壓藥的人，反而比較少同時開胃藥。

正確做法：先用 support 濾掉太罕見的，再用 lift 排序，最後才看 confidence 判斷實用性。

看第 2、3 條

信賴度 0.75 與 0.67 排名很前，但 lift 都小於 1。

★ lift < 1 是負相關

這兩條規則實際上是「開了降血壓藥的人，反而比較少同時開胃藥」——與你想的相反。

正確做法

先用 support 濾掉太罕見的 → 再用 lift 排序 → 最後才看 confidence 判斷實用性。

三個指標的分工

support 管「值不值得看」、lift 管「是不是真的有關」、confidence 管「實際上多可靠」。

五個評估指標的完整定義（含手算範例）

指標	公式	本例數值	怎麼讀
支持度 Support	$P(A \cap B)$	0.60	10 筆中有 6 筆同時出現
信賴度 Confidence	$P(B A) = P(A \cap B)/P(A)$	1.000	開止痛藥的 6 筆，全部都開了胃藥
提升度 Lift	$P(B A)/P(B)$	1.111	★ 只比隨機好 11%，實際關聯很弱
槓桿度 Leverage	$P(A \cap B) - P(A)P(B)$	0.060	實際共現與獨立假設的差距
確信度 Conviction	$(1 - P(B))/(1 - P(B A))$	∞	信賴度為 1 時無定義；一般越大越強

★ 實務只看前三個就夠。後兩個在論文與需要更細緻判斷時才用得上。

兩種頻繁項目集演算法

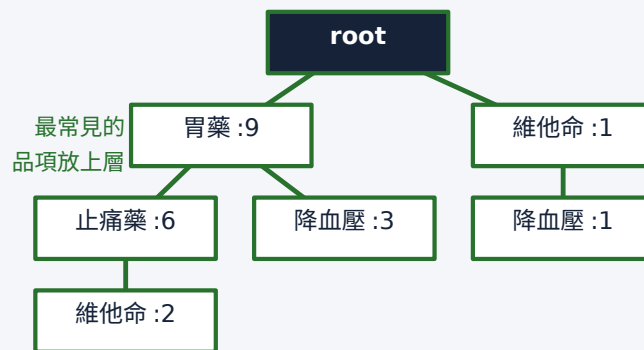
左・Apriori：逐層產生候選並剪枝



★ 核心定理：{A,B} 不頻繁 → {A,B,C} 一定也不頻繁，可直接剪掉

代價：每一層都要重掃一次資料庫（本例 3 次），資料量大時很慢

右・FP-Growth：建一棵前綴樹，只掃兩次



★ 把交易壓縮成一棵前綴樹：共用的前綴只存一次

第 1 次掃：算各品項次數並排序

第 2 次掃：把每筆交易掛上樹

之後頻繁項目集都在樹上挖，不必再碰原始資料

總共只掃資料庫 2 次 → 資料量大時快非常多

Apriori

逐層產生候選並剪枝。核心定理：
如果 {A,B} 不頻繁，{A,B,C} 一定也不頻繁。

Apriori 的代價

要掃描資料庫 N 次（每一層掃一次）。
資料量大時很慢。

★ FP-Growth

把交易壓縮成一棵前綴樹，共用的前綴只存一次。
只掃資料庫 2 次。

實務結論

品項少（< 100）用哪個都行；
品項多或交易量大，一律用 FP-Growth。AI Studio 內建的就是它。

★ 實務結論：品項少（< 100）用哪個都行；品項多或交易量大，一律用 FP-Growth。

AI Studio 內建的 FP-Growth 運算子就是這個演算法；差別只在「要掃幾次資料庫」。

關聯規則在醫護場域的六個應用

應用	把什麼當成「一筆交易」	把什麼當成「品項」	能發現什麼
處方組合分析	一次就診／一張處方	開立的藥品	常見的用藥組合；★ 潛在交互作用風險
共病分析	一位病人	診斷碼（ICD）	哪些疾病常同時出現 → 照護模式設計
檢驗開單模式	一次就診	檢驗項目	★ 是否有不必要的重複開單
衛材耗用	一台手術	使用的衛材	備料組合最佳化
症狀組合	一次諮詢	主訴症狀	哪些症狀常一起出現 → 分診規則參考
照護活動	一個班別	執行的護理活動	工作流程的實際樣態

★ 注意共病分析的「一筆交易」是「一位病人」而非「一次就診」——單位定義錯了，整個分析就沒有意義。

關聯規則的三個常見誤用

① 把關聯當成因果

- ▶ 「開止痛藥 → 開胃藥」不代表止痛藥造成胃藥的需求
- ▶ ★ 很可能是醫師的習慣性搭配，或兩者都由第三個原因造成
- ▶ 關聯規則只告訴你「同時出現」，因果要靠實驗或臨床知識判斷

② 支持度門檻設得太低

- ▶ 設 0.001 會跑出幾萬條規則，其中絕大多數是雜訊
- ▶ ★ 實務起點：support $\geq$ 0.01（1% 的交易）、lift $\geq$ 1.2
- ▶ 規則數量控制在人看得完的範圍（幾十條）才有意義

③ 忽略了「品項粒度」的選擇

- ▶ 用藥品「品項名」還是「成分」還是「ATC 分類」？結果完全不同
- ▶ 粒度太細 → 每個組合都很罕見，跑不出規則
- ▶ ★ 粒度太粗 → 規則太籠統，沒有行動價值。這一步要跟臨床討論

實作 3 | Golf 與購物籃分析

【AI Studio 範例】 Repository ▶ 6. Association Rules/1、2

開啟檔案

Repository ▶ 6. Association Rules/

1. GOLF - FPGrowth
association rules.rmp
2. Market Basket Analysis.rmp
Golf.rmhdf5table
Transactions.rmhdf5table

★ Golf 資料極小，適合
逐條驗算指標

預期結果

- ▶ support 門檻對規則數量的影響是指數級的
- ▶ 兩種排序的前十名會有明顯差異
- ▶ ★ confidence 排名前段的規則，lift 常常很普通
- ▶ 真正有價值的規則通常只有個位數條

操作步驟

- 1 先跑 Golf 範例，資料只有 14 筆——★ 手算驗證一條規則的三個指標
- 2 把 min support 從 0.1 調到 0.5，觀察規則數量如何驟減
- 3 改跑 Market Basket，先用預設參數看跑出幾條規則
- 4 ★ 把結果依 confidence 排序，再依 lift 排序，比較前十名有何不同
- 5 在 confidence 前 10 名中找出 lift 最低的一條（lift 陷阱）
- 6 改變 min lift 門檻為 1.2，看剩下幾條
- 7 挑三條你認為最有行動價值的規則，說明理由

PART 05

時間序列基礎

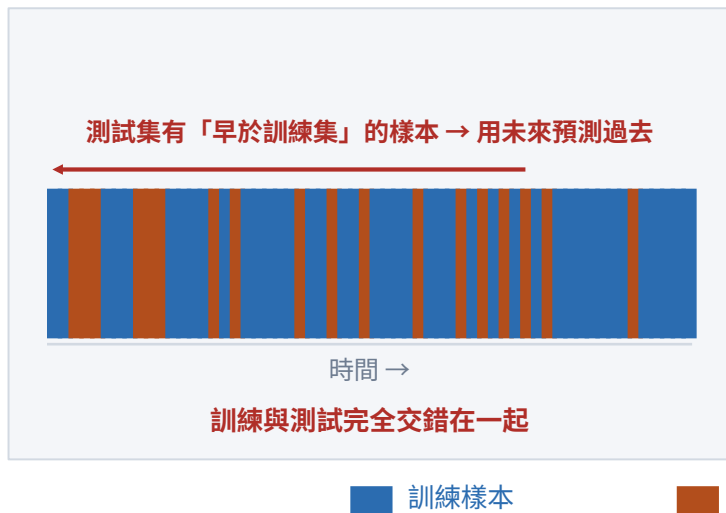
Time Series Fundamentals

當資料有時間順序，前面學過的規則有一半要改寫。
這一節不求完整，但求你不要犯致命錯誤。

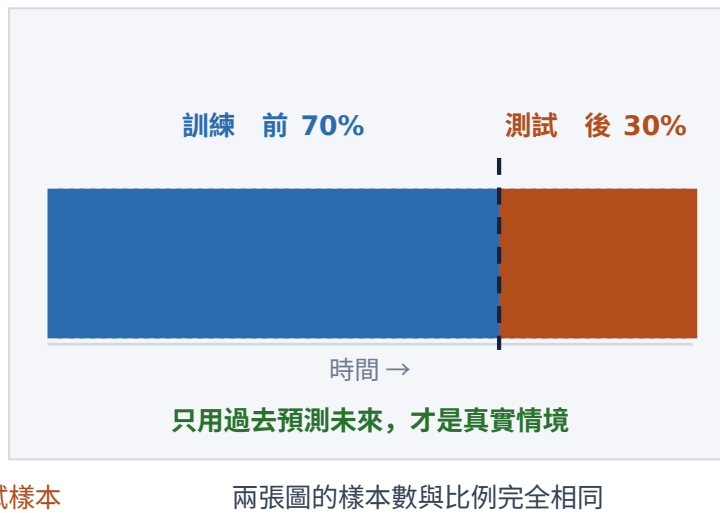
時間序列最容易犯的錯：切分方式

同一份資料的 60 個樣本，由左到右依時間先後排列

左・一般監督式：隨機切分



右・時間序列：依時間切分



兩張圖的樣本數與比例完全相同

★ 資料有時間順序 → M2 用的 Split Data 隨機切分一律不能用。

交叉驗證也要改成 walk-forward（滾動視窗）：每一折的測試集都必須在該折訓練集之後。

左圖的驗證分數會漂亮得離譜——因為模型早就在訓練時看過「未來」了。上線之後才會發現全是假的。

★ 隨機切分是災難

訓練集裡有未來的資料，測試集裡有過去的資料。
這叫資料洩漏，驗證分數會虛高得離譜。

正確做法

依時間切：前面 70% 訓練、後面 30% 測試。
只用過去預測未來，才是真實情境。

交叉驗證也要改

不能用一般 K-Fold。要用 walk-forward（滾動視窗）：
每一折的測試集都在訓練集之後。

★ 一句話

資料有時間順序 → 隨機（shuffled）切分一律不能用。

時間序列與一般監督式學習的五個差異

面向	一般監督式（ M2 ）	時間序列	做錯的後果
切分方式	隨機切分	★ 依時間切	驗證分數虛高，上線就崩
交叉驗證	K-Fold	★ Walk-forward 滾動視窗	同上
樣本獨立性	假設各筆獨立	★ 前後高度相關（自相關）	標準誤估計錯誤
特徵來源	現有欄位	★ 大多要自己造（滯後、移動平均）	沒有特徵可用
評估重點	整體準確率	★ 還要看「預測多遠」仍準確	把短期預測當成長期能力

★ 第四列是實務重點：時間序列的特徵工程比模型選擇重要得多。

滯後特徵：把時間序列變成監督式問題

做完這一步， M2 學過的所有模型就都能用了

特徵工程（時間序列的主戰場）

【原始資料】某急診每日就診人次

日期	人次
2027-03-01	142
2027-03-02	138
2027-03-03	155
2027-03-04	149
2027-03-05	163

【建立滯後特徵】目標：用過去 3 天預測今天

日期	lag_1	lag_2	lag_3	ma_3	dow	is_holiday	y（今日）
2027-03-04	155	138	142	145.0	4	0	149
2027-03-05	149	155	138	147.3	5	0	163

lag_n	= n 天前的人次	（最基本、也最重要的特徵）
ma_3	= 前 3 天的移動平均	（平滑短期波動）
dow	= 星期幾（1-7）	★ 醫療需求有強烈的週週期
is_holiday	= 是否連假	★ 台灣的連假效應非常明顯

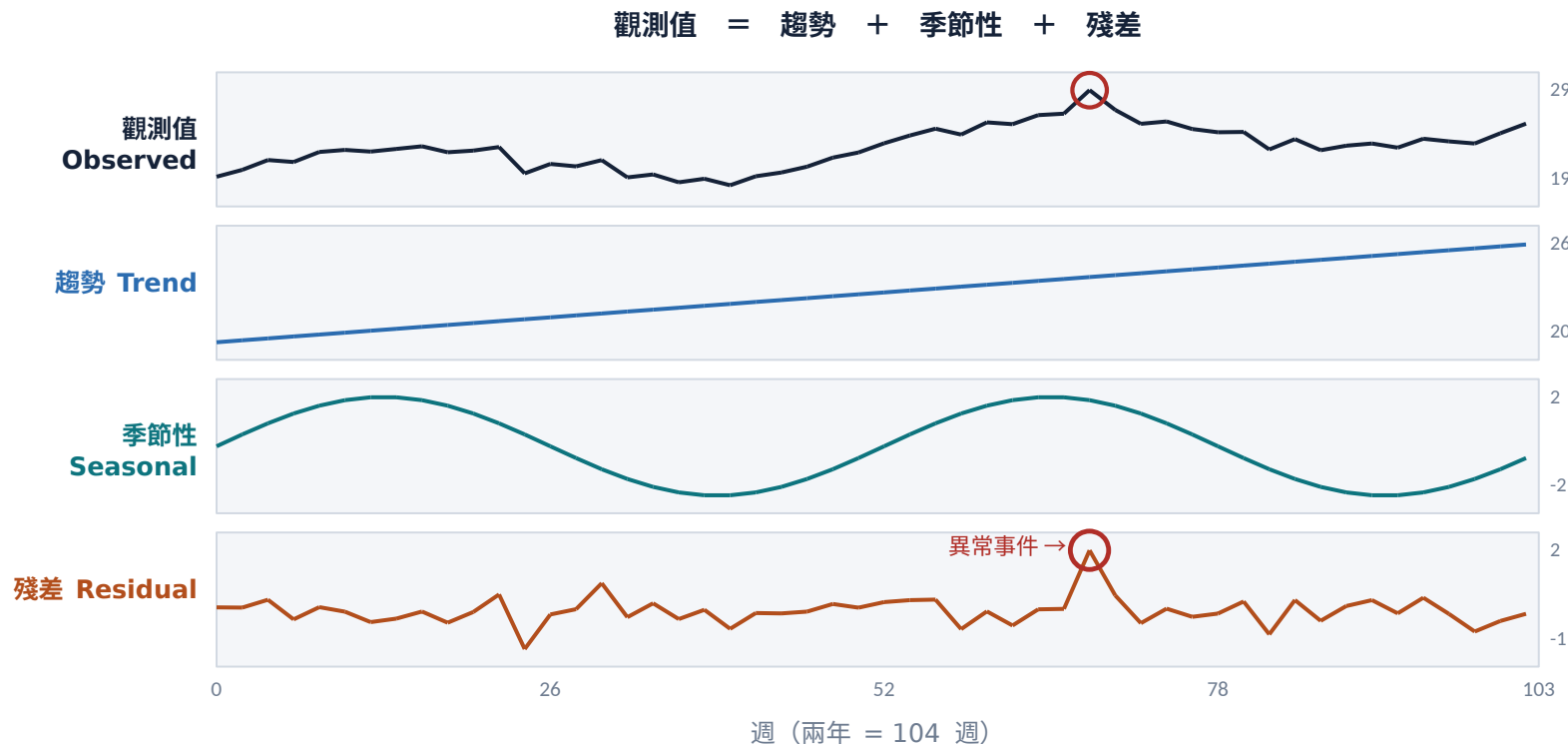
【做完之後】這就是一張普通的表格資料 → 隨機森林、XGBoost 都能直接用

★ 三個一定要注意的地方：

- ① 建完滯後特徵，最前面幾列會有空值（沒有前 3 天）→ 必須刪掉
- ② 絕對不能用「當天或未來」的資訊當特徵 —— 那是最典型的洩漏
例：用「當日住院人數」預測「當日急診人次」→ 上線時你拿不到
- ③ 切分時間點之後，特徵計算不能跨越那個點（滾動視窗要重算）

時間序列分解：把一條線拆成可解釋的成分

範例：急診人次，兩年週資料



★ 趨勢告訴你長期方向、季節性告訴你何時該加派人力、殘差裡的大波動就是異常事件。

第 68 週的殘差突然衝高——單一週是「點異常」，若連續數週偏高才是「集體異常」；分解圖直接把它挑出來了。

趨勢 Trend

長期方向。告訴你「整體是在增加還是減少」。

季節性 Seasonal

規律重複的波動。★ 告訴你何時該加派人力。

殘差 Residual

扣掉前兩者剩下的。★ 裡面的大波動就是異常事件——這裡與 PART 3 的異常偵測接上了。

★ 分解本身就有價值

不一定要做到預測。光是把這三張圖給管理者看，往往就足以支撐排班與資源配置的決策。

時間序列的三個實務提醒

① 先問「真的需要預測嗎」

- ▶ 很多時候管理者要的是「解釋過去」而不是「預測未來」
- ▶ ★ 分解圖 + 同期比較，往往比一個預測值更有用
- ▶ 預測誤差 $\pm 15\%$ 的模型，在排班決策上可能根本用不上

② 基準線一定要建立

- ▶ 最簡單的基準：「明天＝今天」（naive forecast）
- ▶ 季節性基準：「這週三＝上週三」
- ▶ ★ 你的複雜模型如果贏不過這兩個，那就別用了——這很常發生

③ 外部事件要當成特徵

- ▶ 連假、流感季、疫情、大型活動、天氣
- ▶ ★ 台灣的連假效應極強，不放進去模型一定抓不準
- ▶ 這些資訊通常要自己整理成對照表，是很花時間但必要的工作

※ 時間序列的完整方法（ARIMA、Prophet、LSTM）超出本課程範圍；本節目標是讓你不犯致命錯誤，並知道往哪裡進修。

實作 4 | 芝加哥犯罪率時間序列預測

【AI Studio 範例】 Predict Crime Rate.rmp

開啟檔案

```
Repository ▶ 4. Case Studies/  
5. Time Series - Predict  
Crime Rate in Chicago/  
Predict Crime Rate.rmp  
Chicago_Crimes_2001-2017.csv
```

★ 資料橫跨 17 年，
趨勢與季節性都很明顯

預期結果

- ▶ 季節性與 naive 基準孰佳因資料而異
- ▶ ★ 加了滯後特徵的模型不一定贏過季節性基準
- ▶ 若沒贏，表示這份資料的可預測性主要來自季節性
- ▶ 這是很正常也很重要的發現，不是失敗

操作步驟

- 1 先把資料彙總成「每月犯罪件數」，畫出時序圖
- 2 做時間序列分解，觀察趨勢與季節性
- 3 ★ 建立基準線：用「上個月的值」當預測，記錄誤差（MAE）
- 4 ★ 再建一個季節性基準：用「去年同月的值」，記錄誤差
- 5 建立滯後特徵（lag_1、lag_12、月份），跑隨機森林
- 6 依時間切分（前 80% 訓練、後 20% 測試），不要隨機切
- 7 比較三者的 MAE —— 模型有沒有真的贏過基準線？

PART 06

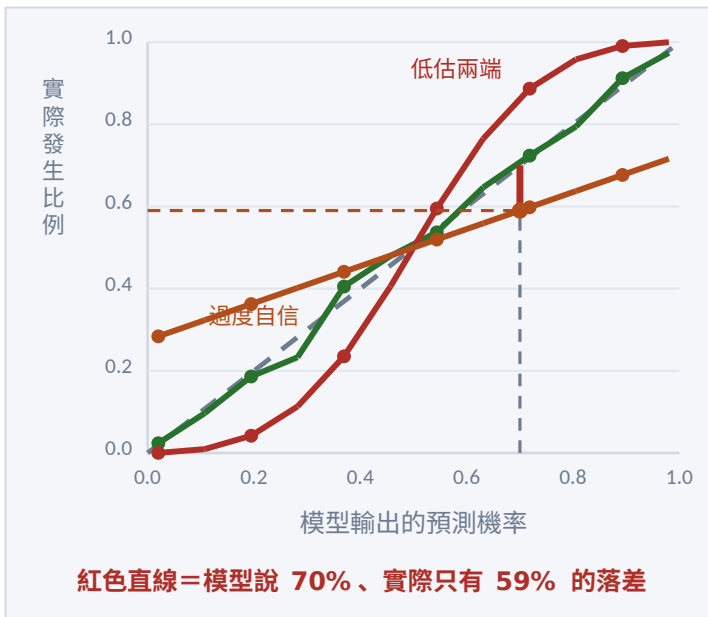
機率校準與代價敏感

Calibration & Cost-Sensitive Learning

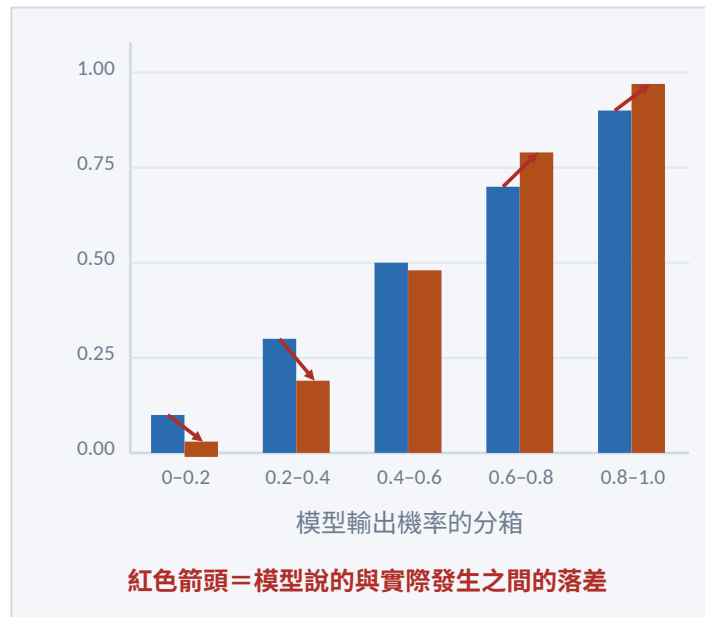
兩個在醫療場域極重要、但在一般課程常被跳過的主題。
它們決定了你的模型能不能真的被拿來用。

機率校準：模型說「70%」的時候，真的有 70% 嗎

左・校準曲線 Calibration Curve



右・未校準模型：兩端偏差最大



— 完美校準（對角線） — 校準良好 — 低估兩端（未調整的隨機森林） — 過度自信（如 Naive Bayes）

★ 為什麼醫療場域特別重要：風險分數要拿來跟臨床決策門檻比對（如「超過 20% 就啟動介入」）。沒校準的機率只能拿來「排序」，不能拿來「當成真實機率解讀」—— 這是最常被誤用的地方。

校準曲線怎麼看

橫軸是模型說的機率、縱軸是實際發生比例。

完美校準應該落在對角線上。

常見的偏差

隨機森林、SVM、AdaBoost 的分數常擠向中間；
Naive Bayes 與過擬合模型常過度自信（Niculescu-Mizil & Caruana, 2005）。

★ 為什麼醫療特別重要

風險分數要拿來跟臨床門檻比對（如「超過 20% 就啟動介入」）。

★ 最常見的誤用

沒校準的機率只能拿來「排序」，不能當成真實機率解讀。

兩種校準方法

方法	作法	適合	注意
Platt Scaling	用邏輯斯迴歸把原始分數映射到機率	★ 樣本少（ < 1000 ）、偏差呈 S 形	假設偏差形狀固定，較不彈性
Isotonic Regression 保序迴歸	學一個單調遞增的映射函數	★ 樣本多（ > 1000 ）、偏差形狀不規則	樣本少時容易過擬合

★ 兩者都必須在「獨立的校準集」上訓練 —— 用訓練集校準等於沒校準。實務做法：切三份（訓練／校準／測試）。

什麼時候你必須做校準

① 要把機率當成「風險值」解讀時

- ▶ 「這位病人再入院風險 35%」——這句話要成立，機率就必須校準
- ▶ ★ 若只是要排序挑出前 100 名高風險，不校準也可以
- ▶ 先問：這個數字會被誰、怎麼使用？

② 要做軟投票或 Stacking 時

- ▶ 各成員的機率若尺度不一致，平均起來就沒有意義
- ▶ ★ 一個過度自信的成員會主導整個集成的結果

③ 要設定決策門檻時

- ▶ 「超過 20% 啟動介入」——這個 20% 必須對應真實的 20%
- ▶ 沒校準的話，你設 20% 實際上可能是 8% 或 45%

★ 反過來說：只需要排名、或門檻由驗證資料調出來時，可以不校準。但畫一張校準曲線來看看仍然值得——它會告訴你這個模型的「性格」。

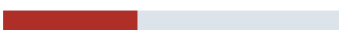
代價敏感學習：把「錯的代價不對稱」放進模型

情境：敗血症早期預警。1000 位病人中實際發生 20 位。

① 一般訓練（追求準確率）

	預測：會	預測：不會
實際會	8	12
實際不會	15	965

↑ 假陰性（漏掉的）

Recall  40%

準確率 97.3% Recall 40%

★ 漏掉 12 位敗血症病人

② 代價敏感訓練

	預測：會	預測：不會
實際會	18	2
實際不會	140	840

↑ 假陰性（漏掉的）

Recall  90%

準確率 85.8% Recall 90%

★ 只漏 2 位，代價是 140 次誤報

把「代價」量化，決策就變得清楚

假設：漏掉一位敗血症的代價 ≈ 200 單位（延誤治療）；一次誤報的代價 ≈ 3 單位（多做一次評估）

$$\textcircled{1} \text{ 總代價} = 12 \times 200 + 15 \times 3 = 2,445 \quad \longrightarrow \quad \textcircled{2} \text{ 總代價} = 2 \times 200 + 140 \times 3 = 820$$

★ 準確率低的那個模型，總代價只有三分之一。這就是為什麼醫療場域不能只看準確率。

問題

一般訓練追求準確率 $\rightarrow 97.3\%$ 很漂亮，但漏掉 12 位敗血症病人（Recall 僅 40%）。

代價敏感訓練

準確率降到 85.8%，但 Recall 提升到 90%，只漏 2 位。

★ 把代價量化就清楚了

總代價 2,445 vs 820。準確率低的那個模型，總代價只有三分之一。

★ 這是醫療 AI 的核心觀念

不能只看準確率。要問「錯在哪一邊、各自的代價是多少」。

三種實作代價敏感的方式

方式	怎麼做	優點	限制
調整類別權重 class_weight	少數類的樣本在損失函數中權重加大	★ 最簡單，多數演算法內建支援	權重比例要自己試
調整決策門檻	訓練照常，預測時把門檻從 0.5 改成 0.2	★ 不用重訓；可依情境動態調整	套臨床既定機率門檻時需先校準
重抽樣 Resampling	過採樣少數類或欠採樣多數類（M2-2 已談）	直觀、與演算法無關	過採樣可能過擬合；欠採樣丟資料
代價矩陣 Cost Matrix	直接把不同錯誤的代價寫進訓練目標	★ 最貼近實際決策；AI Studio：MetaCost	代價數值要跟臨床討論才有意義

★ 最實用的組合：class_weight 訓練 + 校準 + 依代價選門檻。三步都做，模型才真的能用在臨床決策上。

怎麼把「代價」問出來：跟臨床討論的四個問題

這一步不是技術問題，但沒做好整個模型就白建了

需求訪談（這是資料科學家最該練的一項軟技能）

【問題一】漏掉一個（假陰性）會發生什麼事？

✗ 不要問：「Recall 要多少？」——臨床人員不知道你在說什麼

✓ 要問：「如果系統沒示警，但病人真的惡化了，後果是什麼？」

→ 得到：延誤治療的時數、可能的併發症、法律責任

【問題二】誤報一次（假陽性）會發生什麼事？

✓ 要問：「如果系統示警但其實沒事，誰要去做什麼？花多久？」

→ 得到：一次額外評估約 15 分鐘、需要一位護理師

★ 同时要問：「一天最多能承受幾次誤報？」——這決定你的門檻

【問題三】警報疲勞的臨界點在哪？

✓ 要問：「如果十次示警有九次是假的，你們還會理它嗎？」

→ 幾乎所有臨床人員的答案都是「不會」

★ 這比任何統計指標都重要：沒人理的系統，Recall 再高也等於零

【問題四】現在沒有系統的時候，是怎麼做的？

✓ 要問：「目前靠什麼發現？漏掉的比例大概多少？」

→ 得到「人的基準線」——你的模型要贏過的是這個，不是 0

★ 把答案換算成代價比例（例：漏報：誤報 = 200:3），
就能直接算出最佳門檻，而不是憑感覺設 0.5。

AI Studio 範例總覽（皆在 Repository，未複製）

主題	範例檔	資料集	對應章節
集成	Ensemble - bagging vote stacking Adaboost.rmp	Titanic、WineData	PART 1-2
集成	Building Model - processes.rmp	—	PART 2
異常偵測	Anomaly Detection .rmp、Anomaly - outlier detection.rmp	Boiler Sensor	PART 3
關聯規則	1. GOLF - FPGrowth association rules.rmp	Golf	PART 4
關聯規則	2. Market Basket Analysis.rmp	Transactions	PART 4
時間序列	Predict Crime Rate.rmp	Chicago Crimes 2001-2017	PART 5

Take-Home Assignment | M4 進階監督式學習

W8 上課前繳交 · 不另計分（隨堂 Lab）

作業說明

【任務一】集成方法比較 （40%）

資料：Titanic（或自選有標籤的公開資料）

1. 建立基準線：單一決策樹，記錄 10-fold CV 的平均準確率「與標準差」
2. 分別跑 Bagging、AdaBoost、Vote（軟投票）、Stacking
3. 做成一張表：方法 / 平均準確率 / 標準差 / 訓練時間
4. 寫 200 字回答：
 - 哪一種方法主要改善了「平均」？哪一種主要改善了「標準差」？
 - 這個結果符合偏誤－變異的理論預期嗎？

【任務二】異常偵測與門檻設定 （30%）

資料：Boiler Sensor（或自選公開的感測器／生理訊號資料）

1. 先用肉眼標出你認為異常的區段（這是你的對照組）
2. 跑 IQR、LOF、Isolation Forest 三種方法
3. ★ Isolation Forest 的 contamination 分別設 0.01 / 0.05 / 0.10
4. 做成一張表：各方法抓到幾筆、彼此重疊多少、與你的標註吻合多少
5. 寫 150 字：如果每天只有 30 分鐘人力可以覆核，你會怎麼設門檻？

【任務三】關聯規則與 lift 陷阱 （30%）

資料：Transactions（或自選公開的交易／處方資料）

1. 跑 FP-Growth，min support = 0.01
2. 依 confidence 排序取前 10 條；再依 lift 排序取前 10 條
3. ★ 在 confidence 前 10 名中找出 lift 最低的一條規則
4. 寫 150 字：說明這條規則為什麼會誤導人，以及正確的判讀方式

【繳交】PDF 一份，含流程截圖與結果表。檔名：M4_學號_姓名.pdf

本模組重點回顧

- 1 四大主題的共同點：不是「換更強的模型」，而是「換一種看資料的角度」
- 2 ★ Bagging 常降變異； Boosting 常先改善偏誤，但都要靠驗證確認 —— 先診斷症狀再開藥
- 3 集成的前提是多樣性：成員必須「錯得不一樣」，否則白做
- 4 Stacking 的第一層必須用 out-of-fold 預測，否則嚴重洩漏
- 5 ★ 集成的代價是可解釋性；醫療場域要先問「這 0.01 值得嗎」
- 6 異常有三種型態，多數人只想到「點異常」，但情境與集體異常更重要

M4 名詞速查表 集成・異常・關聯・時間序列

(1 / 6)

中文	English	一句話說明
集成學習	Ensemble Learning	組合多個模型的預測以提升表現與穩定性
偏誤	Bias	模型系統性偏離真實關係的程度；高偏誤＝欠擬合
變異	Variance	模型對訓練資料抖動的敏感程度；高變異＝過擬合
多樣性	Diversity	集成成員之間「錯得不一樣」的程度；集成有效的前提
拔靴抽樣	Bootstrap Sampling	有放回的隨機抽樣，每次抽 n 筆
袋外樣本	Out-of-Bag, OOB	未被某次拔靴抽中的樣本，約佔 36.8% ，可當免費驗證集
Bagging	Bootstrap Aggregating	平行訓練多個模型再平均；主要降低變異
隨機森林	Random Forest	Bagging + 每次分裂隨機抽特徵的決策樹集成
Extra Trees	Extremely Randomized Trees	分裂點也隨機取的森林；更快、變異更低

M4 名詞速查表 集成・異常・關聯・時間序列

(2 / 6)

中文	English	一句話說明
Boosting	Boosting	序列訓練，後者修正前者的錯；主要降低偏誤
弱學習器	Weak Learner	僅略優於隨機猜測的模型； Boosting 的基本成員
決策樁	Decision Stump	只有一層的決策樹，常用作 AdaBoost 的弱學習器
AdaBoost	Adaptive Boosting	調整樣本權重讓後續分類器專注於錯誤樣本
梯度提升	Gradient Boosting	以擬合殘差的方式逐步改善；可用於任意可微損失
學習率	Learning Rate / Shrinkage	每棵樹的貢獻比例；與樹的數量成交易關係
早停	Early Stopping	驗證分數不再進步即停止訓練，防止過擬合
逐層生長	Level-wise Growth	整層一起分裂（XGBoost）；樹形對稱、較穩定
逐葉生長	Leaf-wise Growth	每次挑增益最大的葉分裂（LightGBM）；更準更快但易過擬合

M4 名詞速查表 集成・異常・關聯・時間序列

(3 / 6)

中文	English	一句話說明
投票法	Voting	硬投票取多數決；軟投票平均各成員的機率
堆疊法	Stacking	以第二層模型學習如何組合第一層各模型的預測
元學習器	Meta-Learner	Stacking 的第二層模型，通常用簡單模型
折外預測	Out-of-Fold Prediction	以交叉驗證產生的預測值；Stacking 避免洩漏的關鍵
異常偵測	Anomaly Detection	找出與多數樣態明顯不同的資料點
點異常	Point Anomaly	單一資料點明顯偏離
情境異常	Contextual Anomaly	數值本身正常，但在該情境下不正常
集體異常	Collective Anomaly	單點皆正常，但一整段序列不正常
三倍標準差準則	3-Sigma Rule	以 $\mu \pm 3\sigma$ 為界；假設常態分布且易受離群值影響

M4 名詞速查表 集成・異常・關聯・時間序列

(4 / 6)

中文	English	一句話說明
四分位距	Interquartile Range, IQR	$Q3 - Q1$; 以 $Q1 - 1.5IQR$ 、 $Q3 + 1.5IQR$ 為界, 穩健
中位數絕對離差	Median Absolute Deviation, MAD	以中位數為基礎的穩健離散度量
局部離群因子	Local Outlier Factor, LOF	比較自身與鄰居的密度; ≈ 1 正常、 >1 異常
孤立森林	Isolation Forest	以隨機切分所需路徑長度衡量異常; 適合高維
單類別支援向量機	One-Class SVM	學習包住正常資料的邊界
汙染率	Contamination	預期的異常比例; 直接決定會標出幾筆
精確率 @N	Precision@N	分數最高的 N 筆中真正異常的比例; 無標籤時的務實指標
關聯規則	Association Rule	形如 $A \rightarrow B$ 的共現關係
支持度	Support	規則涵蓋的交易比例

M4 名詞速查表 集成・異常・關聯・時間序列

(5 / 6)

中文	English	一句話說明
信賴度	Confidence	$P(B A)$; 在 A 出現的條件下 B 出現的比例
提升度	Lift	$P(B A)/P(B)$; >1 正相關、 $=1$ 獨立、 <1 負相關
槓桿度	Leverage	$P(A \cap B) - P(A)P(B)$; 與獨立假設的差距
確信度	Conviction	$(1 - P(B)) / (1 - P(B A))$; 越大關聯越強
頻繁項目集	Frequent Itemset	支持度高於門檻的品項組合
Apriori	Apriori Algorithm	逐層產生候選並剪枝; 需多次掃描資料庫
FP-Growth	Frequent Pattern Growth	以前綴樹壓縮交易, 只需掃描兩次
購物籃分析	Market Basket Analysis	關聯規則在零售交易上的典型應用
時間序列	Time Series	依時間先後排列的觀測值序列

M4 名詞速查表 集成・異常・關聯・時間序列

(6 / 6)

中文	English	一句話說明
滯後特徵	Lag Feature	以 n 期前的值作為特徵
移動平均	Moving Average	以滑動視窗計算的平均值，用於平滑波動
時間序列分解	Time Series Decomposition	拆解為趨勢、季節性與殘差三成分
自相關	Autocorrelation	序列與自身滯後版本的相關性
滾動驗證	Walk-forward Validation	測試集恆在訓練集之後的時間序列交叉驗證
天真預測	Naive Forecast	以前一期的值作為預測；最基本的基準線